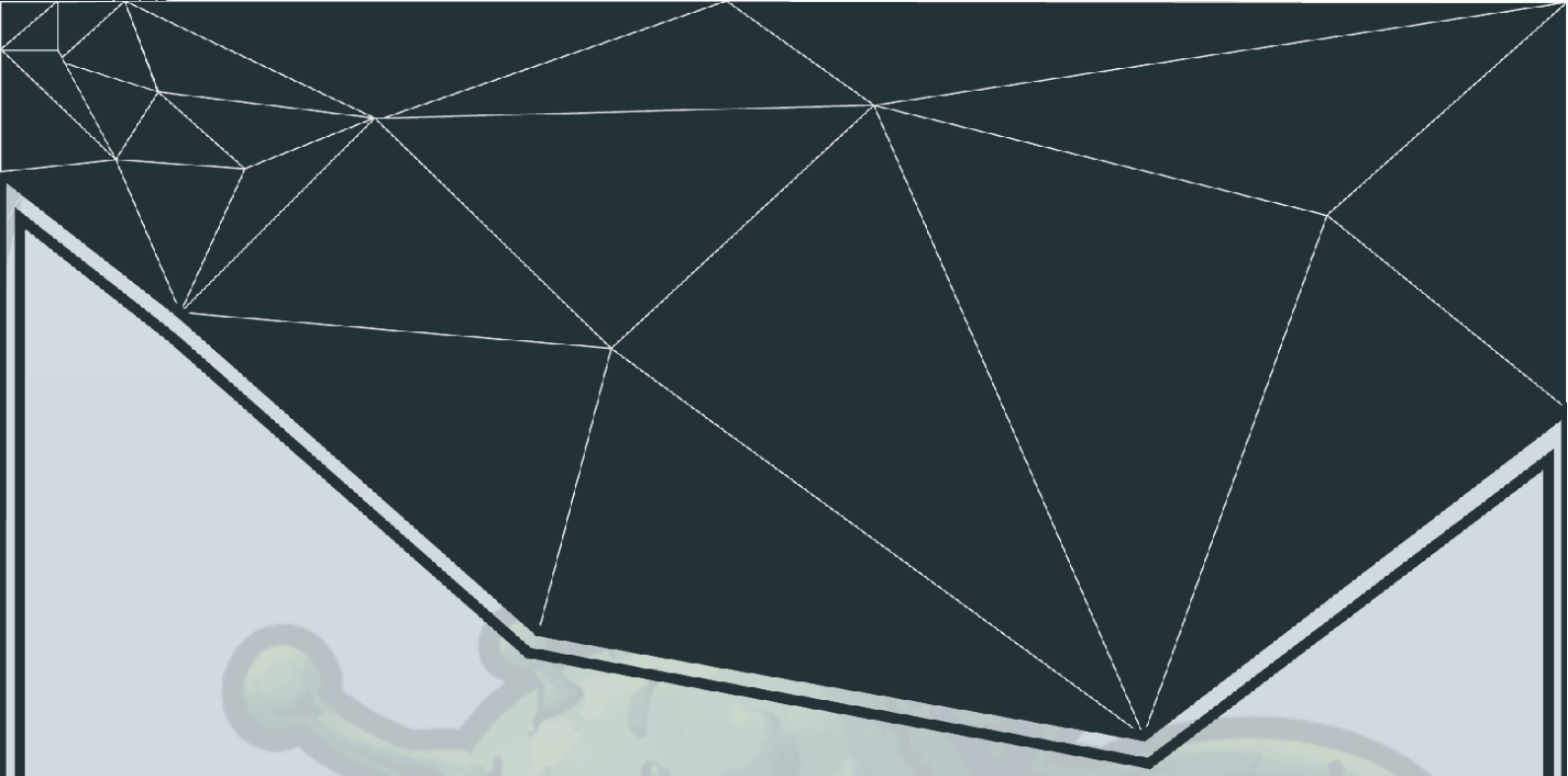




NESHTA

TECHNICAL ANALYSIS REPORT

ZAYOTEM

ZARARLI YAZILIM ÖNLEME VE TERSİNE MÜHENDİSLİK

INDEX

Overview.....	1
neshta.exe Analysis.....	2
Static Analysis.....	2
Dynamic Analysis.....	3
Analysis of the File Under the Temp Directory.....	9
YARA RULE.....	12
MITRE ATTACK TABLE.....	13
Recommendations.....	13
Prepared By.....	14

Overview

The **Neshta** malware family is malicious software targeting Windows systems and is primarily classified as a **file infector**. First identified in the mid-2000s, this malware typically spreads through infected executable files and removable media. Neshta aims to achieve persistence and operate stealthily on compromised systems by disrupting file integrity. .

On systems infected with this malware, it is known to:

- Infect executable (.exe) files,
- Establish persistence by manipulating system startup mechanisms,
- Collect basic system information (computer name, username, operating system details),
- Modify file structures to make detection more difficult,
- Disrupt the normal operation flow of legitimate applications.

neshta.exe Analysis

File Name	Light Tool V9.exe
MD5	10812383882408c1d898b07d29d6e940
SHA256	4cc12d29c4de4d3316ed62add841d20e66bbd5419cd8011e774f937f201be72b
File Type	PE32/EXE

Static Analysis

Property	Value
File Name	C:\Use~1\p\Desktop\neshta.exe
File Type	Portable Executable 32
File Info	Borland Delphi 4.0
File Size	962.50 KB (985600 bytes)
PE Size	40.50 KB (41472 bytes)
Created	Sunday 30 November 2025, 21.18.47
Modified	Sunday 30 November 2025, 18.18.02
Accessed	Sunday 30 November 2025, 21.18.47
MD5	10812383882408C1D898B07D29D6E940
SHA-1	9E0791ADE2278F01659B813F259EDF8FC89EB8B9

Figure 1 – Static Analysis of neshta.exe

When the file is examined using static analysis tools, it is observed that it is a 32-bit PE executable compiled with Delphi. In addition, there is a significant discrepancy between the File Size and the PE Size. This behavior is considered suspicious and may have several possible explanations:

- Packing the code to be unpacked at runtime,
- Padding the file with junk data to evade antivirus detection,
- Embedding another executable, DLL, or shellcode within the file.

	Offset	Boyut	Tip	Kelime
257	0000a0e0	00000044	A	Delphi-the best. Fuck off all the rest. Neshta 1.0 Made in Belarus.

Figure 2 – String Analysis

The strings also confirm that the file belongs to the **Neshta** family.

Dynamic Analysis

Döküm1	Döküm2	Döküm3	Döküm4	Döküm5	İzle 1
Adres	Hex				ASCII
004091E0	2B 5E FE 90	C7 BA E3 57	97 CB 0B 90	B5 E1 D9 6E	+^p.Ç°ăw.Ė..µăŭn
004091F0	CA 12 58 FD	B1 15 2E 9A	74 DB 4E F3	C4 9E 74 93	Ė.xý±...t0N6Ā.t.
00409200	EE E2 68 EE	11 84 8B C0	60 88 B2 F2	54 F8 35 FD	îâhî...Ā`.²òTø5ý
00409210	9C 91 40 00	08 90 40 00	00 00 00 00	00 00 00 00	..@...@.....
00409220	00 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00	

Figure 3 – Encrypted Text Inside the Mutex

The file first creates a **mutex**. The memory region where the mutex name will be written initially contains the encrypted string “+^p.Ç°ăw.Ė.”.

0040708D	85DB	test ebx,ebx
0040708F	7E 10	jle neshta.4070A1
00407091	B8 FF000000	mov eax,FF
00407096	E8 E5B5FFFF	call neshta.402680
00407098	3006	xor byte ptr ds:[esi],al
0040709D	46	inc esi
0040709E	4B	dec ebx
0040709F	75 F0	jne neshta.407091
004070A1	5E	pop esi

Figure 4- XOR Loop

Each character in this text is then **XOR**ed sequentially with the **AL** register value.

00402680	53	push ebx
00402681	31DB	xor ebx,ebx
00402683	6993 08904000 05840800	imul edx,dword ptr ds:[ebx+409008],8088405
0040268D	42	inc edx
0040268E	8993 08904000	mov dword ptr ds:[ebx+409008],edx
00402694	F7E2	mul edx
00402696	89D0	mov eax,edx
00402698	5B	pop ebx
00402699	C3	ret

Figure 5 – The neshta.402680 Function Where the AL Value Is Updated

The **AL** value changes at each iteration of the loop and is determined by the function **neshta.402680**. Within this function, the previous state is updated using the formula **state = state * 0x08088405 + 1**, after which the state value is multiplied by itself and the upper 32 bits are taken. The **AL** portion of this result becomes the next **XOR** key. The initial **AL** value is set at the beginning of the loop.

 Döküm1	 Döküm2	 Döküm3	 Döküm4	 Döküm5	 İzle 1	
Adres	Hex					ASCII
004091E0	73 76 63 68	6F 73 74 2E	63 6F 6D 90	B5 E1 D9 6E	svchost.com.máun	
004091F0	CA 12 58 FD	B1 15 2E 9A	74 DB 4E F3	C4 9E 74 93	É.Xý±...t0NóA.t.	
00409200	EE E2 68 EE	11 84 8B C0	60 88 B2 F2	54 F8 35 FD	îâhî...À`.²òTø5ý	
00409210	9C 91 40 00	08 90 40 00	00 00 00 00	00 00 00 00	..@...@.....	

Figure 6 – svchost.com Written After Decryption

As a result of this process, the string “svchost.com” is written into the mutex.

75A12F6B	8BFF	mov edi,edi	GetModuleFileNameA
75A12F6D	55	push ebp	
75A12F6E	8BEC	mov ebp,esp	
75A12F70	83EC 10	sub esp,10	
75A12F73	57	push edi	
75A12F74	E8 38DFFFFF	call <kernelbase.KernelBaseGetGlobalData>	
75A12F79	8B7D 10	mov edi,dword ptr ss:[ebp+10]	
75A12F7C	8B40 2C	mov eax,dword ptr ds:[eax+2C]	
75A12F7F	64:8B00 18000000	mov ecx,dword ptr [18]	
75A12F86	8D143F	lea edx,dword ptr ds:[edi+edi]	
75A12F89	52	push edx	
75A12F8A	50	push eax	
75A12F8B	8B41 30	mov eax,dword ptr ds:[ecx+30]	
75A12F8E	FF70 18	push dword ptr ds:[eax+18]	
75A12F91	FF15 1810A075	call dword ptr ds:[<rtti!locateheap>]	
75A12F97	8945 FC	mov dword ptr ss:[ebp+4],eax	[ebp-4]:L"C:\\Users\\ o\\Desktop\\neshta.exe"
75A12F9A	85C0	test eax,eax	
75A12F9C	75 11	jne kernelbase.75A12FAE	

Figure 7 – Retrieving the File Name

The file located on the desktop then accesses the name of the running executable via GetModuleFileNameA API.

75A65D8A	8BFF	mov edi,edi	CreateFileA
75A65D8B	55	push ebp	
75A65D8C	8BEC	mov ebp,esp	
75A65D8E	51	push ecx	
75A65D90	51	push ecx	
75A65D92	FF75 08	push dword ptr ss:[ebp+8]	
75A65D95	8045 F8	lea eax,dword ptr ss:[ebp-8]	
75A65D98	50	push eax	eax:"C:\\Users\\...\\AppData\\Local\\Temp\\3582-490\\neshta.exe"
75A65D9C	E8 A1EAF0FF	call <kernel32.Basep88itStringToDynamic>	eax:"C:\\Users\\...\\AppData\\Local\\Temp\\3582-490\\neshta.exe"
75A65D9F	85C0	test eax,eax	esi:"C:\\Users\\...\\AppData\\Local\\Temp\\3582-490\\neshta.exe"
75A65DA2	0F84 24990200	je kernel32.75A8F6F5	[ebp+20]:L"C:\\Users\\...\\AppData\\Local\\Temp\\3582-490\\neshta.exe"
75A65DA5	56	push esi	
75A65DA8	FF75 20	push dword ptr ss:[ebp+20]	
75A65DAB	FF75 1C	push dword ptr ss:[ebp+1C]	
75A65DAD	FF75 18	push dword ptr ss:[ebp+18]	

Figure 8 – File Created Under the Temp Directory

The malware then retrieves the path of the Temp directory using the **GetTempPathA** API and creates a folder named **3582-490** in this location via **CreateDirectoryA**. Since this name is also observed in other **Neshta** samples, it is considered a characteristic indicator of the family. Subsequently, a file named **neshta.exe** is created via **CreateFileA** within this directory.

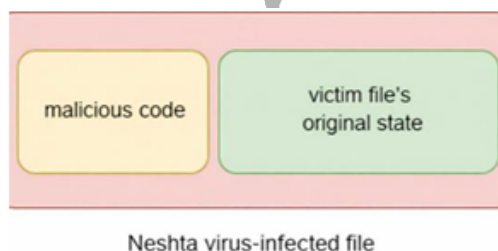


Figure 9 – Structure of Infected File

The malware then performs several operations to restore the original structure of the initially infected file. For this purpose, the file pointer is moved to the end of the section containing the malicious code using **SetFilePointer**. The data following this point is then read and written into the file created under the **Temp** directory using **WriteFile**. As a result of this process, the obtained file becomes an exact copy of the clean file prior to infection.

Address	Disassembly	Comment
75D3103	8BFF mov edi,edi	
75D3104	55 push ebp	
75D3104	8BEC mov ebp,esp	
75D3104	6A 00 push 0	
75D3104	2C 2C push dword ptr ss:[ebp+2C]	[ebp+2C]:L"C:\\Users\\...\\Desktop"
75D3104	28 28 push dword ptr ss:[ebp+28]	
75D3104	24 24 push dword ptr ss:[ebp+24]	
75D3104	20 20 push dword ptr ss:[ebp+20]	
75D3105	1C 1C push dword ptr ss:[ebp+1C]	
75D3105	18 18 push dword ptr ss:[ebp+18]	
75D3105	14 14 push dword ptr ss:[ebp+14]	
75D3105	10 10 push dword ptr ss:[ebp+10]	
75D3105	0C 0C push dword ptr ss:[ebp+0C]	
75D3105	08 08 push dword ptr ss:[ebp+08]	
75D3106	00 00 push 0	
75D3106	E8 CA2B0100 call <kernel32.CreateProcessInternal>	
75D3106	5D pop ebp	
75D3106	C2 2800 ret 28	

Figure 10 – Execution of Clean File

Afterwards, the clean file is executed in order to prevent the user from becoming suspicious.

Address	Disassembly	Comment
75D35D8	8BFF mov edi,edi	
75D35D8	55 push ebp	
75D35D8	8BEC mov ebp,esp	
75D35D8	51 push ecx	
75D35D8	51 push ecx	
75D35D8	08 08 push dword ptr ss:[ebp+08]	
75D35D8	F8 08 lea eax,dword ptr ss:[ebp+08]	
75D35D8	50 push eax	
75D35D8	E8 A1EAF0FF call <kernel32.Basep8BitStringTooby>	
75D35D8	50 50 test eax,eax	
75D35D8	0F84 24990200 je <kernel32.75D5F6F5>	
75D35D8	56 push esi	
75D35D8	20 20 push dword ptr ss:[ebp+20]	
75D35D8	1C 1C push dword ptr ss:[ebp+1C]	
75D35D8	18 18 push dword ptr ss:[ebp+18]	
75D35D8	14 14 push dword ptr ss:[ebp+14]	
75D35D8	10 10 push dword ptr ss:[ebp+10]	
75D35D8	0C 0C push dword ptr ss:[ebp+0C]	
75D35D8	08 08 push dword ptr ss:[ebp+08]	
75D35D8	FC 08 call <kernel32.CreateFileA>	
75D35D8	E8 8B2FFFFF mov esi,eax	
75D35D8	F8 08 lea eax,dword ptr ss:[ebp+08]	
75D35D8	50 push eax	
75D35D8	F15 7005D375 call <kernel32.FreeUnicode>	
75D35D8	86C6 mov eax,esi	
75D35D8	5E pop esi	

Figure 11 – C:\Windows\svchost.com Creation

At this stage, if the malware is executed with administrative privileges, it also creates a file named **svchost[.]com** under **C:\Windows** and writes its raw copy into this file.

Figure 12 - C:\Windows\svchost.com Execution

Then it runs the **svchost[.]com** file.

Figure 13 – Registry Key Creation

Following these operations, a registry key is added so that whenever an executable (.exe) file is launched on the system, svchost[.]com is executed first.

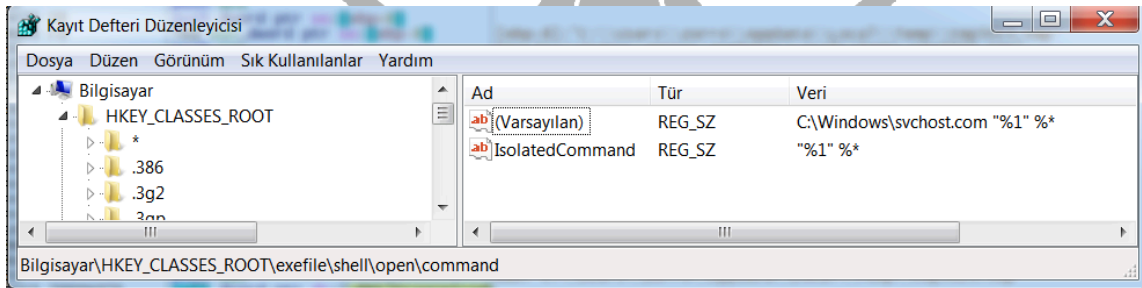


Figure 14 – Registry Key That Executes svchost[.]com Whenever an EXE File Is Launched

This registry key ensures that **svchost[.]com** is automatically executed whenever an .exe file is launched on the system.

Address	Disassembly	Comment
75A6306	8BFF	mov edi,edi
75A6310	55	push ebp
75A6311	8BEC	mov ebp,esp
75A6313	5D	pop ebp
75A6314	EB 05	jmp <JMP.&FindFirstFileA>
75A6316	90	nop
75A6317	90	nop
75A6318	90	nop
75A6319	90	nop
75A631A	90	nop
75A631B	FF25 640AA675	jmp dword ptr ds:[<&FindFirstFileA>]
75A6321	6A 00	push 0
75A6323	68 08020000	push 208
75A6328	8D85 F4DFFFF	lea eax,dword ptr ss:[ebp-20C]
75A632E	50	push eax
75A632F	6A 02	push 2
75A6331	83E3 FC	and ebx,FFFFFFFFC
75A6334	53	push ebx
75A6335	E8 B234FFFF	call <JMP.&GetCurrentProcess>

Figure 15 – File Scanning

The malware then begins overwriting executable files it discovers while traversing all directories on the system. For this purpose, the directories are enumerated using the **FindFirstFileA** and **FindNextFileA** APIs.

Address	Disassembly	Comment
75A6506	8BFF	mov edi,edi
75A6508	55	push ebp
75A6509	8BEC	mov ebp,esp
75A650B	51	push ecx
75A650C	51	push ecx
75A650D	FF75 08	push dword ptr ss:[ebp+8]
75A650E	8D45 F8	lea eax,dword ptr ss:[ebp-8]
75A650F	50	push eax
75A6510	E8 A1EAF7FF	call <kernel32.Basep881tStringToDynamic>
75A6511	85C0	test eax,eax
75A6512	7E 84	je <kernel32.75A8F6F5>
75A6513	56	push esi
75A6514	FF75 20	push dword ptr ss:[ebp+20]
75A6515	FF75 1C	push dword ptr ss:[ebp+1C]
75A6516	FF75 18	push dword ptr ss:[ebp+18]
75A6517	FF75 14	push dword ptr ss:[ebp+14]

Figure 16 – Opening the File for Reading

When an executable file is encountered within a directory, it is opened using the **CreateFile** API for reading.

Address	Disassembly	Comment
75A63F7	6A 0C	push 12
75A63F8	68 4840A675	push kernel32.75A640A8
75A63F9	E8 DBD5FFFF	call <kernel32.75A615CC>
75A63FA	8B7D 14	mov edi,dword ptr ss:[ebp+14]
75A63FB	85FF	test edi,edi
75A63FC	74 03	je <kernel32.75A63FFB>
75A63FD	8327 00	and dword ptr ds:[edi],0
75A63FE	8B75 08	mov esi,dword ptr ss:[ebp+8]
75A63FF	83FE F4	cmp esi,FFFFFFFF4
75A6400	0F84 98B10200	je <kernel32.75A8F1A2>
75A6401	83FE F5	cmp esi,FFFFFFFF5
75A6402	0F84 7E810200	je <kernel32.75A8F18E>
75A6403	83FE F6	cmp esi,FFFFFFFF6
75A6404	0F84 61810200	je <kernel32.75A8F17A>
75A6405	8BC6	mov eax,esi
75A6406	25 03000010	and eax,10000003
75A6407	83F8 03	cmp eax,3
75A6408	0F84 80B10200	je <kernel32.75A8F1B6>
75A6409	FF75 18	push dword ptr ss:[ebp+18]
75A640A	57	push edi
75A640B	FF75 10	push dword ptr ss:[ebp+10]

Figure 17 – Reading the File

The file is then read using the **ReadFile** API to verify whether it contains a valid **MZ** header.

Figure 18 – Opening the File for Writing

The file is then reopened, this time for writing.

Figure 19 – WriteFile Call Stack

[esp+4]	hFile	Yazılacak dosyayı tutan handle
[esp+8]	lpBuffer	Yazılacak verinin bulunduğu adres
[esp+C]	nNumberOfBytesToWrite	Yazılacak byte sayısı (~40,5 KB)
[esp+10]	lpNumberOfBytesWritten	Yazılan byte sayısının yazılacağı adres
[esp+14]	lpOverlapped	NULL → senkron (blocking) yazma

Subsequently, data is written to the file using **WriteFile**. During the WriteFile API call, the parameters observed on the stack indicate that the amount of data written is **0xA200 (~40.5 KB)** bytes, which matches the size of the **svchost[.]com** file.

This process is repeated for all .exe files found on the system. The file is then closed.

In the analysis conducted up to this stage, no evidence was found indicating that the malware directly performs data exfiltration or establishes network communication. However, due to its persistence mechanisms and ability to infect legitimate files, it may enable more extensive and harmful activities when used in conjunction with additional components.

Analysis of the File Under the Temp Directory

File Name	neshta.exe
MD5	562F69643012FCF6D235EBECBBEEA152
SHA256	bb349e47e07df566014d6acfb0525d8e7ec6e2c3288f3f706b5decfb7570273
File Type	PE32/EXE

Property	Value
File Name	C:\Users\...AppData\Local\Temp\3582-490\neshta.exe
File Type	Portable Executable 32
File Info	Microsoft Visual C++ 8
File Size	922.00 KB (944128 bytes)
PE Size	922.00 KB (944128 bytes)

Figure 20 – Static Analysis Output of neshta.exe Located in the Temp Directory

Static analysis of the created executable using various analysis tools shows that the file size and PE size are consistent. The executable also appears to have been developed in C++.

neshta.exe

Icons

String Tables

RCDATA

"SCRIPT" - [lang:0]

Icon Groups

Version Info

Configuration Files

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	Ascii
00000000	A3	48	4B	BE	98	6C	4A	A9	99	4C	53	0A	86	D6	48	7D	EHK%~lJe"IS.+CH}
00000010	41	55	33	21	45	41	30	36	4D	A8	FF	73	24	A7	3C	F6	AU3!EA06M"ys\$S<ö
00000020	7A	12	F1	67	AC	C1	93	E7	6B	43	CA	52	A6	AD	00	00	z]hg-A"çkCER!-..
00000030	E1	BB	3A	21	A5	29	E3	EC	E7	0B	98	2E	40	BD	E1	9A	á»:!¥)äiçd".@%áš

Figure 21 – Contained Autolt Script

Additionally, this new file contains an **Autolt** script.

MD5	ca096b71375b994b5c88a3b7bb72ac65
SHA256	da503a34168b3c21d9884a14b427fef614820a9fa82ae5f76f6e09df5474c2f5



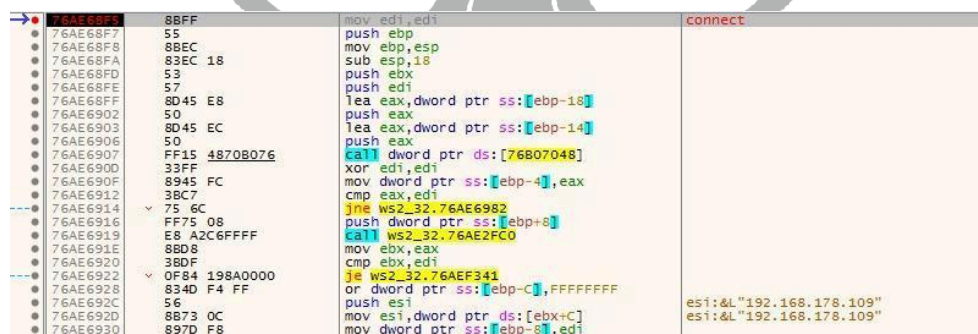
```

{
  "ip": "57.129.46.9",
  "network": "57.129.46.0/23",
  "version": "IPv4",
  "city": "Frankfurt am Main",
  "region": "Hesse",
  "region_code": "HE",
  "country": "DE",
  "country_name": "Germany",
  "country_code": "DE",
  "country_code_iso3": "DEU",
  "country_capital": "Berlin",
  "country_tld": ".de",
  "continent_code": "EU",
  "in_eu": true,
  "postal": "60306",
  "latitude": 50.1162,
  "longitude": 8.6693,
  "timezone": "Europe/Berlin",
  "utc_offset": "+0100",
  "country_calling_code": "+49",
  "currency": "EUR",
  "currency_name": "Euro",
  "languages": "de",
  "country_area": 357021.0,
  "country_population": 82927922,
  "asn": "AS16276",
  "org": "OVH SAS"
}

```

Figure 25 – Target Website Content

When we visit this target, we observe that it is an IP geolocation service providing information such as country, city, ASN, ISP, and timezone based on the IP address. Malware may use this free service, which appears harmless within network traffic, for geo-targeting purposes, allowing it to exhibit different behaviors depending on the victim's location.



Address	Disassembly	Comment
76AE68F6	8BFF	mov edi,edi
76AE68F7	55	push ebp
76AE68F8	8BEC	mov ebp,esp
76AE68FA	83EC 18	sub esp,18
76AE68FD	53	push ebx
76AE68FE	57	push edi
76AE68FF	8D45 E8	lea eax,dword ptr ss:[ebp-18]
76AE6902	50	push eax
76AE6903	8D45 EC	lea eax,dword ptr ss:[ebp-14]
76AE6906	50	push eax
76AE6907	FF15 4870B076	call dword ptr ds:[76B07048]
76AE690D	33FF	xor edi,edi
76AE690F	8945 FC	mov dword ptr ss:[ebp-4],eax
76AE6912	3BC7	cmp eax,edi
76AE6914	75 6C	jne ws2_32.76AE6982
76AE6916	FF75 08	push dword ptr ss:[ebp+8]
76AE6919	E8 A2C6FFFF	call ws2_32.76AE2FC0
76AE691E	8BD8	mov ebx,eax
76AE6920	3BD8	cmp ebx,edi
76AE6922	0F84 198A0000	jbe ws2_32.76AEF341
76AE6928	834D F4 FF	or dword ptr ss:[ebp-C],FFFFFFFF
76AE692C	56	push esi
76AE692D	8B73 0C	mov esi,dword ptr ds:[ebx+C]
76AE6930	897D F8	mov dword ptr ss:[ebp-8],edi

Network Packet Capture:

```

1 192.168.178.109:80 -> 192.168.178.109:80
2 192.168.178.109:80 -> 192.168.178.109:80

```

Figure 26 – Request to Local Target

In the continuation of the analysis, the file continuously generates network traffic toward a local IP address, incrementing the source port number sequentially with each request. This artificially created network noise can be considered a defense evasion technique aimed both at creating unusual network activity on the system.

YARA RULE

```
import "hash"
rule matched_neshta
{
  meta:
    author = "Bilge KELESOGLU - ZAYOTEM"
    description = "Neshta"
    report_date = "04.01.2025"
    file_name = "Light Tool V9.exe"

  strings:
    $neshta = "Neshta"
    $neshtahex = {4E 65 73 68 74 61}
    $original_file = " Light Tool V9"
    $dropped = "svchost.com"
    $mutex = "+^p.Ç°ăW.Ě."
    $directory = "3582-490"
    $domain = "ipapi.co/json"

  condition:
    hash.md5(0, filesize) == "10812383882408c1d898b07d29d6e940" or
    hash.md5(0, filesize) == "562f69643012fcf6d235ebecbbeeA152" or
    hash.md5(0, filesize) == "ca096b71375b994b5c88a3b7bb72ac65" or
    hash.md5(0, filesize) == "dbab013a95d8f47e05e61e3eb9b9da0b" or
    $neshta or $neshtahex or
    (
      $directory and
      $domain and
      ( $mutex or $dropped or $original_file )
    )
}
```

MITRE ATTACK TABLE

Reconnaissance	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	C&C	Exfiltration
T1083 – File and Directory Discovery	T1204.002 – User Execution: Malicious File	T1546.001 – Change Default File Association	T1078 Valid Accounts	T1036 – Masquerading	T1078 – Valid Accounts	T1071.001 – Application Layer Protocol (Web)	-
T1033 – System Owner/User Discovery		T1112 – Modify Registry		T1112 – Modify Registry			
		T1554 – Compromise Host Software Binary		T1027 – Obfuscated Files or Information			

Recommendations

1. At least one up-to-date and reliable antivirus software should be installed on the system.
2. All applications in use should be licensed and kept up to date.
3. Operating system security updates should be applied regularly.
4. Files should not be downloaded from untrusted websites.
5. Software downloaded from the internet should be scanned before use.
6. Unknown applications should not be granted administrative privileges.
7. Multiple spontaneous copies of the same file should be considered suspicious.
8. Comprehensive system scans should be performed in case of unexpected slowdowns or errors.
9. The computer should be restarted at regular intervals to monitor system behavior.
10. Important files should be backed up regularly.

Prepared By

Bilge KELEŞOĞLU

[Linkedin](#)

